

College Management System

9 real tables, 7 phases, PostgreSQL SQL scripts, queries, transactions, and practice exercises

Scope	Students, Professors, Courses, Departments, Hostels, Fees, Results, Enrollments, Allotments
Learning path	Tables, schema, constraints, rows, indexes, joins, subqueries, ACID transactions
Deliverables	Schema design, DDL, seed data, updates/deletes, indexes, SELECT queries, transactions, practice

How to use this PDF
Read Phase 1 first, run the Phase 2 and Phase 3 SQL in a fresh PostgreSQL database, then complete the exercises after each phase. The practice section at the end is designed as a mini lab test.

Project Roadmap

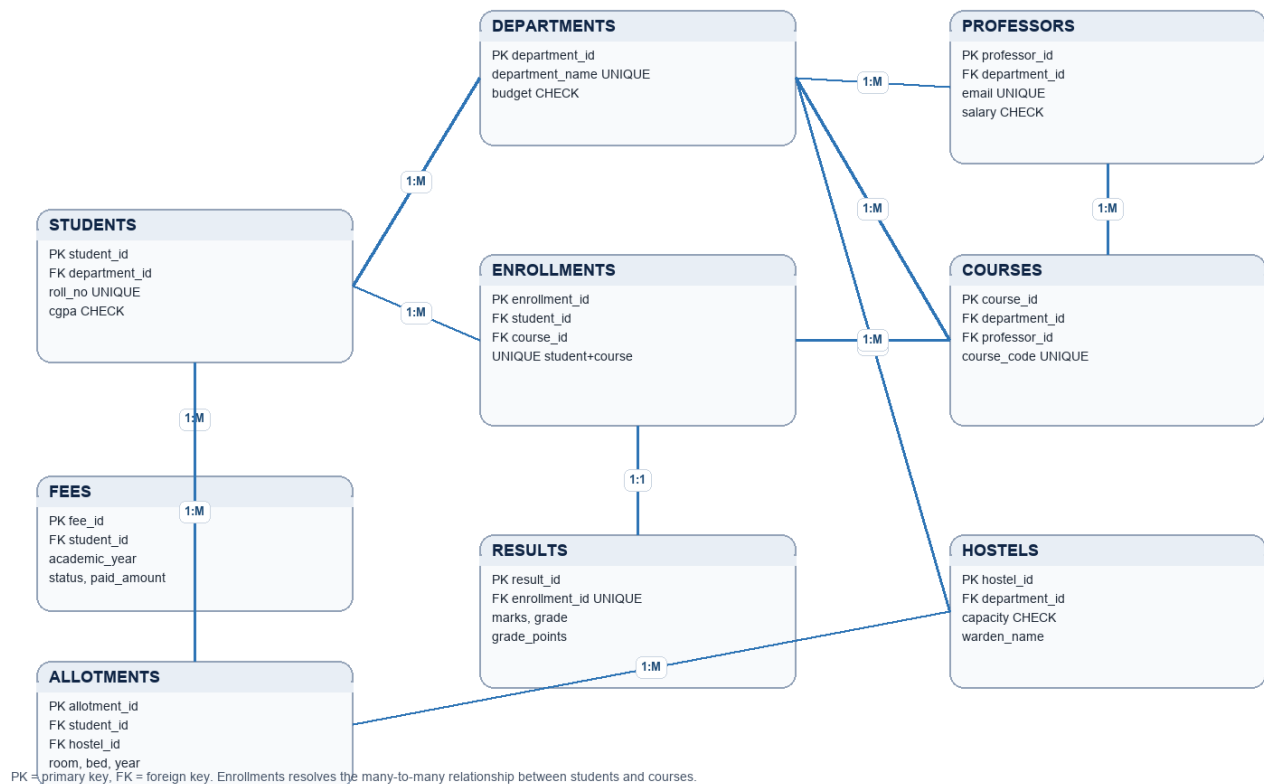
This project teaches relational database design through one coherent college management system. Each phase adds a practical layer, so the schema moves from paper design to executable SQL and then to real query behavior.

Phase	Build	Concepts Practiced
1. Design	Identify 9 entities, relationships, and schema diagram.	Tables, schema, relationships
2. Create	Write CREATE TABLE statements with columns, keys, constraints, and cascades.	Columns, PK, FK, constraints
3. Populate	Insert realistic data, then update and delete rows.	Rows, INSERT, UPDATE, DELETE
4. Indexes	Create 7 indexes and compare EXPLAIN plans.	Index, query planning
5. Queries	Write 9 SELECT queries from simple joins to a 5-table profile.	JOINS, GROUP BY, subqueries
6. Transactions	Run 3 real transactions with COMMIT, rollback logic, and SAVEPOINT.	ACID
7. Practice	Complete task blocks and self-assessment.	All concepts

Phase 1 - Design

The design uses exactly 9 real tables. Two tables, enrollments and allotments, act as relationship tables that convert practical college workflows into clean relational structure.

College Management System - ER Schema



Entity	Purpose	Main Relationships
Departments	Academic units such as Computer Science or Electronics.	One department has many students, professors, courses, and optional hostels.
Students	Learners admitted to departments.	One student has many enrollments, fees, results through enrollments, and hostel allotments.
Professors	Faculty who teach courses.	One professor can teach many courses.
Courses	Subjects offered by departments.	Many students take many courses through enrollments.
Enrollments	Bridge between students and courses.	Connects student to course and stores enrollment status.
Results	Exam outcome for an enrollment.	One result belongs to one enrollment.
Fees	Yearly fee demand and payment status.	One student can have one fee record per academic year.
Hostels	Residential buildings.	One hostel can have many allotments.
Allotments	Bridge between students and hostels.	Stores room, bed, year, and active/vacated status.

Relationship	Cardinality	Reason
Department to Students	1 to many	Each student belongs to one department; a department admits many students.
Department to Professors	1 to many	Each professor is assigned to one department.
Department to Courses	1 to many	Courses are owned by departments.
Professor to Courses	1 to many	A professor can teach multiple courses; a course has one current professor.
Student to Course	many to many	Resolved by enrollments.
Enrollment to Result	1 to 0/1	A result is entered only after an exam.
Student to Fees	1 to many	One fee row per academic year.
Student to Hostel	many over time	Resolved by allotments.

Phase 1 Exercises

- 1 Draw the same schema without looking at the diagram and label every primary key and foreign key.
- 2 Explain why enrollments is required instead of storing course_id directly in students.
- 3 Choose one relationship and write the business rule that justifies its cardinality.

Phase 2 - Create Tables

The DDL below uses six core constraint types: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT. The cascade rules are deliberately mixed so you can learn when to cascade, restrict, or set a child value to NULL.

Constraint	Where It Appears	Why It Matters
PRIMARY KEY	Every table	Identifies each row uniquely.
FOREIGN KEY	Child tables	Protects relationships between tables.
NOT NULL	Required columns	Prevents incomplete records.
UNIQUE	Roll numbers, emails, course codes, room assignments	Prevents duplicates.
CHECK	CGPA, marks, fee amount, status values	Rejects invalid values.
DEFAULT	created_at, dates, status, cgpa, paid_amount	Supplies safe starting values.

```
-- Phase 2: PostgreSQL DDL for the 9-table College Management System.
-- Drop child tables first so the script can be re-run during practice.
DROP TABLE IF EXISTS results CASCADE;
DROP TABLE IF EXISTS enrollments CASCADE;
DROP TABLE IF EXISTS allotments CASCADE;
DROP TABLE IF EXISTS fees CASCADE;
DROP TABLE IF EXISTS courses CASCADE;
DROP TABLE IF EXISTS hostels CASCADE;
DROP TABLE IF EXISTS students CASCADE;
DROP TABLE IF EXISTS professors CASCADE;
DROP TABLE IF EXISTS departments CASCADE;
```

```
CREATE TABLE departments (
  department_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  department_name VARCHAR(80) NOT NULL UNIQUE,
  office_phone VARCHAR(15) UNIQUE,
  budget NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (budget >= 0),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE professors (
  professor_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  department_id INT NOT NULL,
  professor_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  hire_date DATE NOT NULL DEFAULT CURRENT_DATE,
  salary NUMERIC(10,2) NOT NULL CHECK (salary >= 30000),
  designation VARCHAR(40) NOT NULL
  CHECK (designation IN ('Assistant Professor', 'Associate Professor',
    'Professor')),
  CONSTRAINT fk_professor_department
  FOREIGN KEY (department_id)
  REFERENCES departments(department_id)
  ON UPDATE CASCADE
```

```
ON DELETE RESTRICT
);

CREATE TABLE students (
  student_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  roll_no VARCHAR(20) NOT NULL UNIQUE,
  department_id INT NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  gender VARCHAR(10) NOT NULL CHECK (gender IN ('Female', 'Male', 'Other')),
  dob DATE NOT NULL CHECK (dob < DATE '2012-01-01'),
  admission_year SMALLINT NOT NULL CHECK (admission_year BETWEEN 2020 AND 2030),
  cgpa NUMERIC(4,2) NOT NULL DEFAULT 0 CHECK (cgpa BETWEEN 0 AND 10),
  email VARCHAR(120) NOT NULL UNIQUE,
  CONSTRAINT fk_student_department
  FOREIGN KEY (department_id)
  REFERENCES departments(department_id)
  ON UPDATE CASCADE
  ON DELETE RESTRICT
);

CREATE TABLE courses (
  course_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  department_id INT NOT NULL,
  professor_id INT,
  course_code VARCHAR(15) NOT NULL UNIQUE,
  course_title VARCHAR(100) NOT NULL,
  credits SMALLINT NOT NULL DEFAULT 3 CHECK (credits BETWEEN 1 AND 6),
  semester SMALLINT NOT NULL CHECK (semester BETWEEN 1 AND 8),
  CONSTRAINT fk_course_department
  FOREIGN KEY (department_id)
  REFERENCES departments(department_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  CONSTRAINT fk_course_professor
  FOREIGN KEY (professor_id)
  REFERENCES professors(professor_id)
  ON UPDATE CASCADE
  ON DELETE SET NULL
);

CREATE TABLE hostels (
  hostel_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  department_id INT,
  hostel_name VARCHAR(80) NOT NULL UNIQUE,
  capacity INT NOT NULL CHECK (capacity > 0),
  warden_name VARCHAR(100) NOT NULL,
  hostel_for VARCHAR(10) NOT NULL CHECK (hostel_for IN ('Female', 'Male', 'Any')),
  CONSTRAINT fk_hostel_department
  FOREIGN KEY (department_id)
  REFERENCES departments(department_id)
  ON UPDATE CASCADE
  ON DELETE SET NULL
);

CREATE TABLE enrollments (
  enrollment_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  enrollment_date DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(20) NOT NULL DEFAULT 'Enrolled'
  CHECK (status IN ('Enrolled', 'Dropped', 'Completed')),
  CONSTRAINT uq_student_course UNIQUE (student_id, course_id),
  CONSTRAINT fk_enrollment_student
  FOREIGN KEY (student_id)
  REFERENCES students(student_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  CONSTRAINT fk_enrollment_course
```

```

FOREIGN KEY (course_id)
REFERENCES courses(course_id)
ON UPDATE CASCADE
ON DELETE CASCADE
);

CREATE TABLE results (
result_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
enrollment_id INT NOT NULL UNIQUE,
exam_type VARCHAR(20) NOT NULL DEFAULT 'Final'
CHECK (exam_type IN ('Midterm', 'Final', 'Supplementary')),
marks NUMERIC(5,2) NOT NULL CHECK (marks BETWEEN 0 AND 100),
grade VARCHAR(2) NOT NULL CHECK (grade IN ('O', 'A+', 'A', 'B+', 'B', 'C', 'F')),
grade_points NUMERIC(4,2) NOT NULL CHECK (grade_points BETWEEN 0 AND 10),
result_date DATE NOT NULL DEFAULT CURRENT_DATE,
CONSTRAINT fk_result_enrollment
FOREIGN KEY (enrollment_id)
REFERENCES enrollments(enrollment_id)
ON UPDATE CASCADE
ON DELETE CASCADE
);

CREATE TABLE fees (
fee_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
student_id INT NOT NULL,
academic_year VARCHAR(9) NOT NULL CHECK (academic_year LIKE '____-____'),
amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
due_date DATE NOT NULL,
paid_amount NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (paid_amount >= 0),
status VARCHAR(20) NOT NULL DEFAULT 'Pending'
CHECK (status IN ('Pending', 'Partially Paid', 'Paid', 'Overdue')),
paid_on DATE,
CONSTRAINT ck_fee_paid_amount CHECK (paid_amount <= amount),
CONSTRAINT uq_student_fee_year UNIQUE (student_id, academic_year),

```

```

CONSTRAINT fk_fee_student
FOREIGN KEY (student_id)
REFERENCES students(student_id)
ON UPDATE CASCADE
ON DELETE CASCADE
);

CREATE TABLE allotments (
allotment_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
student_id INT NOT NULL,
hostel_id INT NOT NULL,
room_no VARCHAR(10) NOT NULL,
bed_no VARCHAR(10) NOT NULL,
academic_year VARCHAR(9) NOT NULL CHECK (academic_year LIKE '____-____'),
allotted_on DATE NOT NULL DEFAULT CURRENT_DATE,
vacated_on DATE,
status VARCHAR(20) NOT NULL DEFAULT 'Active'
CHECK (status IN ('Active', 'Vacated')),
CONSTRAINT ck_allotment_dates CHECK (vacated_on IS NULL OR vacated_on >=
allotted_on),
CONSTRAINT uq_student_year_allotment UNIQUE (student_id, academic_year),
CONSTRAINT uq_room_bed_year UNIQUE (hostel_id, room_no, bed_no, academic_year),
CONSTRAINT fk_allotment_student
FOREIGN KEY (student_id)
REFERENCES students(student_id)
ON UPDATE CASCADE
ON DELETE CASCADE,
CONSTRAINT fk_allotment_hostel
FOREIGN KEY (hostel_id)
REFERENCES hostels(hostel_id)
ON UPDATE CASCADE
ON DELETE CASCADE
);

```

Phase 2 Exercises

- 1 Find one CHECK constraint and intentionally try to insert invalid data to observe the error.
- 2 Change one ON DELETE CASCADE rule to ON DELETE RESTRICT and explain how behavior changes.
- 3 Add a UNIQUE constraint to prevent two professors from sharing the same office phone.
- 4 Write a short note explaining why paid_amount must be checked against amount.

Phase 3 - Populate and Modify Data

This seed script creates 5 departments, 8 students, 5 professors, 7 courses, enrollment rows, exam results, hostel allotments, and fee records. It then performs realistic UPDATE and DELETE operations.

```
-- Phase 3: realistic seed data. Explicit IDs keep classroom examples readable.
INSERT INTO departments (department_id, department_name, office_phone, budget) VALUES
(1, 'Computer Science', '040-1001', 2500000.00),
(2, 'Electronics', '040-1002', 1800000.00),
(3, 'Mechanical Engineering', '040-1003', 2200000.00),
(4, 'Civil Engineering', '040-1004', 1700000.00),
(5, 'Business Administration', '040-1005', 1600000.00);

INSERT INTO professors
(professor_id, department_id, professor_name, email, hire_date, salary, designation)
VALUES
(1, 1, 'Dr. Anita Rao', 'anita.rao@college.edu', '2016-07-15', 92000.00, 'Associate Professor'),
(2, 2, 'Dr. Vikram Sen', 'vikram.sen@college.edu', '2014-06-01', 98000.00, 'Professor'),
(3, 3, 'Prof. Nirmal Shah', 'nirmal.shah@college.edu', '2019-08-05', 76000.00, 'Assistant Professor'),
(4, 4, 'Dr. Farah Khan', 'farah.khan@college.edu', '2015-01-10', 94000.00, 'Professor'),
(5, 5, 'Prof. Kavya Menon', 'kavya.menon@college.edu', '2020-11-20', 72000.00, 'Assistant Professor');

INSERT INTO students
(student_id, roll_no, department_id, full_name, gender, dob, admission_year, cgpa, email) VALUES
(1, 'CS24-001', 1, 'Asha Reddy', 'Female', '2005-04-12', 2024, 8.42, 'asha.reddy@student.edu'),
(2, 'CS24-002', 1, 'Rohit Mehta', 'Male', '2004-08-21', 2024, 7.95, 'rohit.mehta@student.edu'),
(3, 'EC24-001', 2, 'Neha Iyer', 'Female', '2005-01-09', 2024, 8.10, 'neha.iyer@student.edu'),
(4, 'ME23-001', 3, 'Karthik Menon', 'Male', '2004-02-17', 2023, 7.40, 'karthik.menon@student.edu'),
(5, 'CE23-001', 4, 'Sara Khan', 'Female', '2004-11-04', 2023, 8.01, 'sara.khan@student.edu'),
(6, 'BA24-001', 5, 'Divya Nair', 'Female', '2005-06-30', 2024, 8.33,
```

```

'divya.nair@student.edu'),
(7, 'CS23-003', 1, 'Imran Ali', 'Male', '2003-12-18', 2023, 8.70,
'imran.ali@student.edu'),
(8, 'EC23-002', 2, 'Meera Thomas', 'Female', '2004-05-25', 2023, 8.60,
'meera.thomas@student.edu');

INSERT INTO courses
(course_id, department_id, professor_id, course_code, course_title, credits, semester)
VALUES
(1, 1, 1, 'CS101', 'Database Management Systems', 4, 3),
(2, 1, 1, 'CS201', 'Data Structures', 4, 3),
(3, 2, 2, 'EC101', 'Digital Electronics', 3, 2),
(4, 3, 3, 'ME101', 'Engineering Mechanics', 4, 2),
(5, 4, 4, 'CE101', 'Surveying', 3, 2),
(6, 5, 5, 'BA101', 'Principles of Management', 3, 1),
(7, 1, 1, 'CS301', 'Operating Systems', 4, 4);

INSERT INTO hostels
(hostel_id, department_id, hostel_name, capacity, warden_name, hostel_for) VALUES
(1, 1, 'Maple Hostel', 120, 'Mrs. Lakshmi Das', 'Female'),
(2, 1, 'Cedar Hostel', 140, 'Mr. Ramesh Pillai', 'Male'),
(3, 2, 'Lotus Hostel', 100, 'Mrs. Priya Varma', 'Female'),
(4, 5, 'Orchid Hostel', 90, 'Mr. Joseph Mathew', 'Any');

INSERT INTO enrollments
(enrollment_id, student_id, course_id, enrollment_date, status) VALUES
(1, 1, 1, '2025-07-01', 'Completed'),
(2, 1, 2, '2025-07-01', 'Completed'),
(3, 1, 7, '2026-01-05', 'Enrolled'),
(4, 2, 1, '2025-07-01', 'Completed'),
(5, 2, 2, '2025-07-01', 'Completed'),
(6, 3, 3, '2025-07-01', 'Completed'),
(7, 3, 1, '2026-01-05', 'Enrolled'),
(8, 4, 4, '2025-07-01', 'Completed'),

(9, 4, 6, '2026-01-05', 'Enrolled'),
(10, 5, 5, '2025-07-01', 'Completed'),
(11, 6, 6, '2025-07-01', 'Completed'),
(12, 7, 1, '2025-07-01', 'Completed'),
(13, 7, 7, '2026-01-05', 'Enrolled'),
(14, 8, 3, '2025-07-01', 'Completed'),
(15, 8, 7, '2026-01-05', 'Enrolled');

INSERT INTO results
(result_id, enrollment_id, exam_type, marks, grade, grade_points, result_date) VALUES
(1, 1, 'Final', 88.50, 'A+', 9.00, '2025-12-20'),
(2, 2, 'Final', 91.00, 'O', 10.00, '2025-12-20'),
(3, 4, 'Final', 78.00, 'B+', 8.00, '2025-12-20'),
(4, 5, 'Final', 82.25, 'A', 8.50, '2025-12-20'),
(5, 6, 'Final', 84.00, 'A', 8.50, '2025-12-20'),
(6, 8, 'Final', 73.00, 'B+', 8.00, '2025-12-20'),
(7, 10, 'Final', 86.50, 'A+', 9.00, '2025-12-20'),
(8, 11, 'Final', 90.00, 'O', 10.00, '2025-12-20'),
(9, 12, 'Final', 92.00, 'O', 10.00, '2025-12-20'),
(10, 14, 'Final', 89.00, 'A+', 9.00, '2025-12-20');

INSERT INTO fees
(fee_id, student_id, academic_year, amount, due_date, paid_amount, status, paid_on)
VALUES
(1, 1, '2025-2026', 85000.00, '2025-08-10', 85000.00, 'Paid', '2025-08-01'),
(2, 2, '2025-2026', 85000.00, '2025-08-10', 42500.00, 'Partially Paid', '2025-08-05'),
(3, 3, '2025-2026', 78000.00, '2025-08-10', 78000.00, 'Paid', '2025-08-03'),
(4, 4, '2025-2026', 80000.00, '2025-08-10', 0.00, 'Overdue', NULL),
(5, 5, '2025-2026', 76000.00, '2025-08-10', 76000.00, 'Paid', '2025-07-28'),
(6, 6, '2025-2026', 70000.00, '2025-08-10', 35000.00, 'Partially Paid', '2025-08-07'),
(7, 7, '2025-2026', 85000.00, '2025-08-10', 85000.00, 'Paid', '2025-08-02'),
(8, 8, '2025-2026', 78000.00, '2025-08-10', 0.00, 'Pending', NULL);

INSERT INTO allotments

```



```
(allotment_id, student_id, hostel_id, room_no, bed_no, academic_year, allotted_on,
status) VALUES
(1, 1, 1, 'A-101', 'B1', '2025-2026', '2025-07-15', 'Active'),
(2, 2, 2, 'C-205', 'B2', '2025-2026', '2025-07-15', 'Active'),
(3, 3, 3, 'L-110', 'B1', '2025-2026', '2025-07-15', 'Active'),
(4, 6, 4, 'O-012', 'B1', '2025-2026', '2025-07-15', 'Active'),
(5, 7, 2, 'C-210', 'B1', '2025-2026', '2025-07-15', 'Active'),
(6, 8, 3, 'L-118', 'B2', '2025-2026', '2025-07-15', 'Active');

-- UPDATE examples.
UPDATE professors
SET salary = salary * 1.08
WHERE department_id = (
SELECT department_id FROM departments WHERE department_name = 'Computer Science'
);

UPDATE students
SET cgpa = 8.55
WHERE roll_no = 'CS24-001';

UPDATE fees
SET paid_amount = amount,
status = 'Paid',
paid_on = CURRENT_DATE
WHERE student_id = (SELECT student_id FROM students WHERE roll_no = 'CS24-002');

-- DELETE example. Deleting an enrollment also deletes its result because of ON DELETE
CASCADE.
DELETE FROM enrollments
WHERE enrollment_id = 8;
```

Phase 3 Exercises

- 1 Insert one new student into Civil Engineering and enroll the student in CE101.
- 2 Update all overdue fees to Pending after extending the due date by 15 days.
- 3 Delete one course and observe which enrollment and result rows are removed by cascade.
- 4 Write a SELECT statement to verify the salary raise for Computer Science professors.

Phase 4 - Indexes and EXPLAIN

Indexes speed up common lookup, join, filter, and sort patterns. On this small classroom dataset, timings may be tiny; the important lesson is reading whether PostgreSQL can use an index scan when the table grows.

```
-- Phase 4: seven indexes for common query patterns.
CREATE INDEX idx_students_department_id
ON students(department_id);

CREATE INDEX idx_professors_department_id
ON professors(department_id);

CREATE INDEX idx_courses_department_semester
ON courses(department_id, semester);

CREATE INDEX idx_enrollments_student_id
ON enrollments(student_id);

CREATE INDEX idx_enrollments_course_id
ON enrollments(course_id);

CREATE INDEX idx_results_grade_points
ON results(grade_points DESC);

CREATE INDEX idx_fees_status_due_date
ON fees(status, due_date);

-- Test before and after creating indexes.
EXPLAIN ANALYZE
SELECT s.roll_no, s.full_name, d.department_name
FROM students s
JOIN departments d ON d.department_id = s.department_id
WHERE s.department_id = 1;

EXPLAIN ANALYZE
SELECT f.status, f.due_date, s.roll_no, s.full_name
FROM fees f
JOIN students s ON s.student_id = f.student_id
WHERE f.status IN ('Pending', 'Overdue')
```

```
ORDER BY f.due_date;
```

Query Pattern	Before Index	After Index	Learning Point
Students by department	Sequential scan is likely on tiny data.	Index scan can appear as rows grow.	Foreign key filters are good index candidates.
Courses by department and semester	Sort/filter may scan courses.	Composite index supports both filter and order.	Column order matters.
Fees by status and due date	Filter then sort.	Index can locate status and return due order faster.	Useful for admin dashboards.
Enrollments by student	Scan enrollments.	Index lookup for one student's courses.	JOIN tables need indexes on both FK columns.

Phase 4 Exercises

- 1 Run EXPLAIN ANALYZE before creating indexes and save the plan text.
- 2 Create the indexes and run the same EXPLAIN ANALYZE commands again.
- 3 Drop idx_fees_status_due_date and explain what changed in the fee report plan.

Phase 5 - Progressive SELECT Queries

The queries move from simple JOINS to aggregation, subqueries, and a five-table academic profile. They are written so students can run each one immediately after Phase 3.

```
-- Query 1: Students with departments.
SELECT s.roll_no, s.full_name, d.department_name, s.cgpa
FROM students s
JOIN departments d ON d.department_id = s.department_id
ORDER BY d.department_name, s.roll_no;

-- Query 2: Courses with their professor and department.
SELECT c.course_code, c.course_title, d.department_name, p.professor_name
FROM courses c
JOIN departments d ON d.department_id = c.department_id
LEFT JOIN professors p ON p.professor_id = c.professor_id
ORDER BY c.course_code;

-- Query 3: Students currently staying in hostels.
SELECT s.roll_no, s.full_name, h.hostel_name, a.room_no, a.bed_no
FROM allotments a
JOIN students s ON s.student_id = a.student_id
JOIN hostels h ON h.hostel_id = a.hostel_id
WHERE a.status = 'Active'
ORDER BY h.hostel_name, a.room_no;

-- Query 4: Fee report with balance amount.
SELECT s.roll_no, s.full_name, f.academic_year, f.amount,
f.paid_amount, (f.amount - f.paid_amount) AS balance, f.status
FROM fees f
JOIN students s ON s.student_id = f.student_id
ORDER BY f.status, balance DESC;

-- Query 5: Enrollment list for one student.
SELECT s.roll_no, s.full_name, c.course_code, c.course_title, e.status
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE s.roll_no = 'CS24-001'
```

```
ORDER BY c.semester, c.course_code;

-- Query 6: Average marks by course using GROUP BY.
SELECT c.course_code, c.course_title, ROUND(AVG(r.marks), 2) AS avg_marks,
COUNT(r.result_id) AS result_count
FROM results r
JOIN enrollments e ON e.enrollment_id = r.enrollment_id
JOIN courses c ON c.course_id = e.course_id
GROUP BY c.course_code, c.course_title
ORDER BY avg_marks DESC;

-- Query 7: Students whose CGPA is above the college average.
SELECT roll_no, full_name, cgpa
FROM students
WHERE cgpa > (SELECT AVG(cgpa) FROM students)
ORDER BY cgpa DESC;

-- Query 8: Pending or overdue fees.
SELECT s.roll_no, s.full_name, f.status, f.due_date,
CURRENT_DATE - f.due_date AS days_after_due_date
FROM fees f
JOIN students s ON s.student_id = f.student_id
WHERE f.status IN ('Pending', 'Overdue')
ORDER BY f.due_date;

-- Query 9: Full five-table student academic profile.
SELECT s.roll_no, s.full_name, d.department_name,
c.course_code, c.course_title,
r.marks, r.grade, r.grade_points
FROM students s
JOIN departments d ON d.department_id = s.department_id
JOIN enrollments e ON e.student_id = s.student_id
JOIN courses c ON c.course_id = e.course_id
LEFT JOIN results r ON r.enrollment_id = e.enrollment_id
```

```
WHERE s.roll_no = 'CS24-001'
ORDER BY c.semester, c.course_code;
```

Phase 5 Exercises

- 1 Modify Query 1 to show only students with CGPA greater than 8.00.
- 2 Modify Query 4 to display only students who still have a balance.
- 3 Write a GROUP BY query that counts students per department.
- 4 Write a subquery that returns students enrolled in Database Management Systems.
- 5 Extend Query 9 to include hostel name and fee status.

Phase 6 - Transactions and ACID

Transactions make multiple SQL statements succeed or fail as one logical unit. The examples below demonstrate atomicity, consistency, durability, and a SAVEPOINT-based partial rollback.

ACID Term	Meaning in This Project
Atomicity	A new student, enrollment, and fee row are committed together or not at all.
Consistency	Fee status and paid amount stay valid because constraints and transaction logic agree.
Isolation	FOR UPDATE prevents two clerks from changing the same fee row at the same time.
Durability	After COMMIT, the payment or result remains stored.

```
-- Transaction 1: student enrollment with atomicity.
BEGIN;

INSERT INTO students
(roll_no, department_id, full_name, gender, dob, admission_year, cgpa, email)
VALUES
('CS26-009', 1, 'Tanvi Sharma', 'Female', '2006-03-14', 2026, 0,
'tanvi.sharma@student.edu');

INSERT INTO enrollments (student_id, course_id)
SELECT student_id, 1
FROM students
WHERE roll_no = 'CS26-009';

INSERT INTO fees (student_id, academic_year, amount, due_date)
SELECT student_id, '2026-2027', 85000.00, DATE '2026-08-10'
FROM students
WHERE roll_no = 'CS26-009';

COMMIT;

-- Transaction 2: fee payment with consistency and durability.
BEGIN;

SELECT fee_id, amount, paid_amount, status
FROM fees
WHERE fee_id = 4
FOR UPDATE;

UPDATE fees
SET paid_amount = amount,
status = 'Paid',
paid_on = CURRENT_DATE
WHERE fee_id = 4;
```

```

COMMIT;

-- Transaction 3: exam result entry using SAVEPOINT and partial rollback.
BEGIN;

SAVEPOINT before_result_entry;

INSERT INTO results
(enrollment_id, exam_type, marks, grade, grade_points, result_date)
VALUES
(3, 'Final', 66.00, 'B', 7.00, CURRENT_DATE);

-- Suppose the mark sheet was checked again and the marks were entered wrong.
ROLLBACK TO SAVEPOINT before_result_entry;

INSERT INTO results
(enrollment_id, exam_type, marks, grade, grade_points, result_date)
VALUES
(3, 'Final', 86.00, 'A+', 9.00, CURRENT_DATE);

COMMIT;

```

Phase 6 Exercises

- 1 Run Transaction 1, then ROLLBACK instead of COMMIT and prove no partial rows remain.
- 2 Modify Transaction 2 to handle a partial payment and set status to Partially Paid.
- 3 Create a SAVEPOINT before entering three result rows, roll back one wrong row, then commit the rest.

Phase 7 - Practice Lab

Use these task blocks as a final hands-on lab. Each block has a clear output, so it can be graded or self-checked.

Block	Task
1. Design	Add a sports club feature on paper only. Identify 3 possible entities and their relationships.
2. DDL	Add a CHECK constraint that prevents negative hostel capacity in your own words, then compare with the provided DDL.
3. Insert	Insert one professor, one course, and two enrollments for that course.
4. Update	Give a 5 percent salary raise to professors who teach more than one course.
5. Delete	Delete a student and list all child rows that should disappear because of cascade rules.
6. Index	Create an index for searching students by full_name and test it with EXPLAIN.
7. Join	Show every student, department, hostel room, and fee status in one query.
8. Aggregate	Find average CGPA by department and sort highest first.
9. Subquery	List students who scored above the average marks in CS101.
10. Transaction	Write a transaction that allots a hostel room and rejects duplicate bed assignment.

15-Item Self-Assessment

- 1 Can I identify all 9 tables and describe each table's purpose?
- 2 Can I name the primary key of every table?
- 3 Can I explain the difference between PRIMARY KEY and UNIQUE?
- 4 Can I explain why enrollments is a bridge table?

- 5 Can I write a FOREIGN KEY with ON DELETE CASCADE?
- 6 Can I choose between CASCADE, RESTRICT, and SET NULL for a business rule?
- 7 Can I insert parent rows before child rows without FK errors?
- 8 Can I update fee status without violating CHECK constraints?
- 9 Can I predict what child rows are deleted when a parent row is deleted?
- 10 Can I create an index for a WHERE filter?
- 11 Can I read whether EXPLAIN uses a sequential scan or index scan?
- 12 Can I write INNER JOIN and LEFT JOIN queries correctly?
- 13 Can I use GROUP BY with aggregate functions?
- 14 Can I write a subquery in a WHERE clause?
- 15 Can I use BEGIN, COMMIT, ROLLBACK, and SAVEPOINT in a real workflow?

Completion target

A strong student should be able to run the DDL and seed data, answer all self-assessment items, and write at least five new SELECT queries without copying the examples.